

Bus Booking Frontend Review:

- Frontend is page based not ng component based.
- Each page/component have one HTML file and one TS file with their features
- There is one core folder which includes Models, guards, services interceptors.
- core/services/auth.service.ts handles login, register, logout, token state, role state, and JWT decoding.
- core/services/api.service.ts contains backend API calls for routes, trips, bookings, payment, tickets, admin, operator, and profile features.
- core/interceptors/auth.interceptor.ts automatically attaches the bearer token to HTTP requests.
- core/interceptors/logging.interceptor.ts is used for request/response debugging.
- core/guards/auth.guard.ts prevents unauthenticated users from opening protected pages.
- core/guards/role.guard.ts allows only specific roles to access pages.
- app.routes.ts defines page navigation and role access rules.
- core/models.ts gives typed interfaces for API request and response data.

Merits of the AI-Generated Frontend

- The project is well organized with separate services, guards, models, and components.
- Authentication and API calls are centralized, making the code easier to manage.
- JWT interceptor automatically adds tokens to requests, reducing repeated code.
- Role-based access is implemented for User, Operator, and Admin functionalities.
- Reactive forms and TypeScript interfaces improve validation and code reliability.
- The application already supports a complete booking flow and is ready for demonstrations.

Demerits and Improvements Needed

- API URLs are hardcoded and should be moved to environment files.

- JWT tokens are stored in localStorage, which has security risks.
- Error handling and some API call logic can be made more reusable and optimized.
- Some subscriptions need proper cleanup to avoid memory leaks.
- Token expiry handling and search optimization can be improved.
- More unit testing and code cleanup are needed before final deployment.